

이번 채팅은 **전체 트렌드 목록 조회 설계 전용**으로 두고, 지금은 구현으로 넘어가지 않겠습니다. 먼저 GitHub의 **remote dev 최신 상태**, 최신 가이드라인, Taxonomy 확정안, 실제 ORM/Alembic, 현재 Frontend 구조를 확인했습니다.

현재 기준점은 dev HEAD ****8c934018b644934aa188501140fecb16d0213051****이며, 2026-09-21의 **feat-interest-edit** 병합 커밋입니다. 따라서 말씀하신 관심사 조회/수정 Frontend와 **InterestEditScreen**까지 병합된 상태가 실제 **dev**에서도 확인됩니다.

1. 현재 코드/DB 기준점 점검 결과

영역	현재 dev에서 확인된 사실	이번 설계에 미치는 영향
Trend API	<code>trend_router.py</code> , <code>trend_service.py</code> , <code>trend_repository.py</code> , <code>trend_schema.py</code> 없음	기존 구현과의 하위 호환 부담 없이 처음부터 계약을 설계 가능
통합 Router	<code>auth</code> , <code>category</code> , <code>interest</code> , <code>health</code> 만 연결됨	<code>/api/trends</code> 는 신규 Router 연결 필요
Frontend Trend	<code>features/trend</code> 자체가 아직 없음	API/Hook/Type/Card 구조를 처음부터 정리 가능
Main	<code>MainPlaceholderScreen</code> 상태	실제 트렌드 UI 책임은 아직 동결되지 않음
Navigation	<code>Main</code> , <code>InterestEdit</code> 존재	Trend 목록/상세 Navigation은 아직 미정
TanStack Query	<code>@tanstack/react-query ^5.101.2</code>	일반 Query 또는 Infinite Query 모두 현재 구조에서 설계 가능
Trend 상태	<code>ACTIVE</code> , <code>HIDDEN</code>	일반 사용자 조회 상태 규칙을 명확히 할 수 있음
Trend 시간	<code>first_collected_at</code> , <code>last_collected_at</code> , <code>updated_at</code>	무엇을 " 최신 "이라고 부를지 제품 결정 필요
Trend 기본 내용	<code>title</code> , nullable <code>summary</code> , nullable <code>thumbnail_url</code>	기본 카드 구성 가능
Category	Trend ↔ Category M:N	카드 한 개에 여러 Category가 존재 가능
Rank	Trend당 플랫폼별 다수의 시계열 Snapshot	단일 <code>rank</code> 라는 값은 현재 DB에 존재하지 않음
Source	Trend당 여러 <code>trend_sources</code>	단일 <code>source</code> 역시 현재 DB에 존재하지 않음
AI	Trend당 버전별 여러 분석 결과	목록에서 어떤 분석 버전을 사용할지 정해야 함

특히 실제 **trends** ORM과 Alembic에는 다음 인덱스가 이미 있습니다.

```
INDEX(status, last_collected_at)
```

따라서 **ACTIVE + last_collected_at 기준 최신 조회**는 현재 스키마가 이미 잘 지원하는 조회 패턴입니다. 다만 이것이 제품적으로 올바른 "최신"의 정의인지는 별개의 결정입니다.

또 하나 중요한 점은 Taxonomy v1.0.1입니다. 현재 확정안은 Trend를 세부분류에 연결하고, 부모 대분류를 중복 저장하지 않는 것으로 확정되어 있습니다.

신작 모바일 RPG

trend_category_map

→ 모바일 게임

저장하지 않음

→ 게임 + 모바일 게임

그리고 **is_primary**는 DB에는 남아 있지만 **MVP에서 사용처를 만들지 않는** 것으로 이미 확정되어 있습니다. 따라서 카드에 Category 하나만 임의로 골라서 보여 주는 구현은 현재 확정안만으로는 근거가 없습니다.

2. 이번 설계에서 이미 거의 확정 가능한 부분

아래는 제품 판단을 새로 요구하기보다는 기존 구조를 그대로 따르면 되는 후보입니다.

Trend 상태는 ACTIVE만 일반 목록에 노출하는 것이 현재 **TrendStatus**의 "조회 가능 상태" 의미와 일치합니다. **HIDDEN**까지 반환할 이유가 생기지 않는 한 별도 질문을 만들 필요가 없어 보입니다.

데이터가 0건인 경우도 특정 Trend 하나를 찾는 것이 아니라 Collection 조회이므로,

200 OK

```
{
  "items": []
}
```

계열로 처리하는 것이 기존 관심사 Collection 조회 정책과도 일관됩니다. 404 대상으로 보기는 어렵습니다.

Backend 책임 역시 기존 프로젝트 원칙상 큰 선택지가 없습니다.

Router

→ HTTP/API 계약

Service

→ Trend 목록 Use Case 및 제품 규칙

```
Repository
→ SQLAlchemy 조회 / 정렬 / Pagination / Join

Database
```

Frontend도 현재 관심사 기능의 패턴을 유지한다면,

```
Screen
→ Trend Query Hook
→ Trend API Function
→ authenticated/public API Client

TrendCard
→ 전달받은 데이터 표현
```

이 기본 책임 분리는 유지하는 편이 자연스럽습니다. 특히 TrendCard가 직접 API를 호출하거나 Query를 소유하는 방향은 피하는 것이 좋습니다.

다만 어떤 API Client를 사용하는지, Query를 useQuery로 할지 useInfiniteQuery로 할지, Repository가 어느 Join까지 수행해야 하는지는 지금부터 정할 제품 규칙에 따라 달라집니다.

3. Design First Pass

이번에는 제가 답을 먼저 동결하지 않겠습니다. 아래 질문에 현재 생각나는 수준까지만 답해주시면 됩니다. 기술적인 정답을 맞히실 필요는 없고, "A가 더 맞는 것 같다 / 잘 모르겠다" 정도도 충분합니다.

1. GET /api/trends는 로그인한 사용자만 볼 수 있어야 할까요?

선택	의미	영향
A. 인증 필요	로그인 후 앱 내부 최신 이슈 기능	authenticatedApiClient, 공통 401 적용
B. 인증 불필요	로그인 전에도 전체 Trend 공개 가능	publicApiClient, 향후 Guest UX 가능

현재 Trend Leader에는 Guest Navigation 자체가 없고 Main도 인증 사용자에게만 열립니다. 그래서 현재 구조만 보면 A가 단순하지만, "전체 최신 이슈는 로그인 없이도 볼 수 있어야 한다"는 제품 의도가 있다면 B가 맞을 수 있습니다.

질문: 사용자 입장에서 Trend Leader의 최신 이슈를 보려면 반드시 로그인이 필요하다고 생각하시나요?

2. GET /api/trends 화면은 최종적으로 Main인가요, 별도의 최신 이슈 화면인가요?

이 결정은 생각보다 큼니다.

A안

```
Main
→ GET /api/trends
→ 전체 최신 Trend
```

먼저 전체 Trend를 Main으로 만들고, 이후 `/recommended`가 생기면 Main 구조를 다시 확장합니다.

B안

```
Main
→ 향후 GET /api/trends/recommended

Latest / 전체 트렌드 화면
→ GET /api/trends
```

초기 기획 방향처럼 "Main = 개인화, 전체 최신 이슈 = 별도 기능"을 유지합니다.

현재 `MainPlaceholderScreen`은 말 그대로 Placeholder여서 어느 쪽도 코드상 동결되어 있지는 않습니다.

질문: 최종 제품에서도 Main의 핵심은 맞춤 추천이어야 한다고 보시나요, 아니면 우선 전체 최신 Trend가 Main이어도 괜찮다고 보시나요?

3. "최신 Trend"에서 **최신**은 정확히 무엇을 뜻해야 할까요?

현재 DB에는 서로 다른 의미의 시간이 세 개 있습니다.

기준	의미	예시
<code>first_collected_at</code>	처음 발견된 Trend	오늘 처음 등장한 이슈
<code>last_collected_at</code>	가장 최근 다시 관측된 Trend	일주일 전 시작했지만 지금도 활발한 이슈
Rank Snapshot	현재 플랫폼에서 강한 Trend	최근 수집 여부보다 순위가 중요

예를 들어 오래전에 처음 생긴 "**아이폰 신제품**" Trend가 오늘 다시 강하게 수집되었다면:

```
first_collected_at = 9월 10일
last_collected_at  = 9월 21일
```

입니다.

질문: "전체 최신 트렌드"는 **최근 처음 등장한 것**을 보여주려는 기능인가요, 아니면 **지금도 최근에 관측되고 있는 것**을 보여주려는 기능인가요?

이 답에 따라 기본 정렬축을 확정하겠습니다.

4. Category 필터를 `GET /api/trends` 첫 버전부터 제공할까요?

이 항목은 Taxonomy 확정안에서 **Trend** 목록 설계 단계에서 결정하도록 의도적으로 남겨 둔 사항입니다.

A안

```
GET /api/trends
```

전체 최신 Trend만 조회하고 Category는 카드 표시용으로만 사용합니다.

B안

```
GET /api/trends?category_id=...
```

Category별 최신 Trend도 같은 Endpoint에서 조회할 수 있게 합니다.

B를 선택하면 다시 중요한 의미가 생깁니다.

```
게임(대분류) 선택
→ 모바일 게임
→ PC 게임
→ 콘솔 게임
→ e스포츠
→ ...
전체 하위 Category Trend를 조회할 것인가?

모바일 게임(세부분류) 선택
→ 모바일 게임만 조회할 것인가?
```

질문: 이번 MVP 전체 Trend 화면에서 사용자가 Category를 눌러 필터링하는 기능까지 필요하다고 생각하시나요?

5. 목록은 어떤 식으로 더 불러오기를 원하시나요?

먼저 UX를 정하고 Pagination 기술을 고르는 편이 좋습니다.

A. 모바일 피드 방식

```
20개 조회
↓
아래로 스크롤
↓
다음 20개
↓
계속 추가
```

이 경우 `useInfiniteQuery`와 Cursor Pagination을 검토할 이유가 커집니다.

B. 페이지 방식

1페이지 / 2페이지 / 3페이지

page + size 방식이 단순합니다.

트렌드는 사용자가 보는 동안에도 새 데이터가 위에 추가될 수 있는 성격이라 Cursor와 Offset의 동작 차이도 이후 검토해야 합니다.

질문: Trend Leader에서는 일반 SNS처럼 스크롤하며 계속 불러오는 목록을 생각하고 계셨나요, 아니면 페이지 개념이 있어도 괜찮나요?

6. Trend Card에서 Category를 어느 수준까지 보여주고 싶으신가요?

한 Trend가 여러 세부분류에 연결될 수 있습니다.

예:

갤럭시 AI 기능 화제
→ IT/디지털 > 인공지능
→ IT/디지털 > 모바일·스마트폰

가능한 표현은 대략 다음과 같습니다.

방식	카드 표시
A	인공지능, 모바일·스마트폰
B	IT/디지털 > 인공지능, IT/디지털 > 모바일·스마트폰
C	IT/디지털만 표시
D	세부분류를 최대 2~3개만 표시하고 나머지는 생략

현재 is_primary를 쓰지 않기로 했으므로 "대표 Category 하나"를 임의로 고르는 방식은 별도 규칙 없이는 사용하기 어렵습니다.

질문: 실제 모바일 Card에서는 어떤 수준이 가장 보기 좋고 정보가 충분하다고 생각하시나요?

7. 전체 Trend Card에서 Rank는 정말 핵심 정보인가요?

여기가 이번 설계에서 가장 조심해야 할 부분 중 하나입니다.

현재 DB에는:

Trend A
└ GOOGLE ranking = 3

```
└─ YOUTUBE ranking = 7
└─ SNS ranking = 2
```

처럼 **플랫폼별 순위**가 존재할 수 있습니다.

즉 현재 구조에는:

```
{
  "rank": 3
}
```

같은 "**Trend 자체의 단일 순위**"가 없습니다.

가능한 정책은 다음과 같습니다.

선택 의미

- | | |
|---|----------------------------|
| A | 전체 최신 목록에서는 Rank를 표시하지 않는다 |
| B | 플랫폼별 Rank를 함께 표시한다 |
| C | 특정 대표 플랫폼 Rank만 표시한다 |
| D | 종합 Rank를 새로 정의한다 |

D는 단순 Response 설계가 아니라 **새로운 Ranking Domain Rule**이므로 이번 범위를 꽤 크게 넓힙니다.

질문: 사용자가 전체 최신 목록을 볼 때 "**몇 위인가**"가 꼭 보여야 한다고 생각하시나요? 아니면 이번 첫 버전에서는 최신성 중심으로 가도 괜찮을까요?

8. Source는 목록 Card에 어느 정도 보여야 할까요?

한 Trend에는 여러 Source가 존재할 수 있습니다.

```
Trend
└─ Google source 1
└─ YouTube source 1
└─ YouTube source 2
└─ SNS source 1
```

따라서 "**source** 하나를 반환"하려면 대표 출처 선정 정책이 새로 필요합니다.

선택지는 예를 들어:

- A. 목록에서는 GOOGLE / YOUTUBE / SNS 같은
출처 플랫폼 종류만 표시

- B. 대표 Source 하나를 골라 제목/URL까지 표시
- C. 목록에서는 Source를 표시하지 않고
상세 화면에서만 제공

정도로 나뉩니다.

질문: Card에서 사용자가 "어디에서 나온 Trend인지"까지 바로 알아야 한다고 보시나요? 그렇다면 플랫폼 이름 정도면 충분한가요, 실제 원문 링크까지 필요할까요?

9. AI 분석을 목록에 얼마나 노출하고 싶으신가요?

현재 데이터 모델에는 명확히 두 종류의 설명이 있습니다.

```
trends.summary
→ 기본/수집 기반 요약
→ NULL 가능

trend_ai_analyses.one_line_summary
→ AI가 만든 분석 버전별 한줄 요약
```

가능한 Card 계약은:

```
A.
title
+ trends.summary

B.
title
+ 최신 AI one_line_summary
  없으면 trends.summary fallback

C.
목록은 title 중심
AI 내용은 상세에서만 표시
```

입니다.

특히 이번 범위에는 **실제 AI Provider 연동이 없기 때문에**, B를 선택하더라도 테스트에서는 미리 준비한 AI 분석 데이터로 검증해야 합니다.

질문: Trend Leader의 차별점을 생각했을 때 AI 한줄 요약은 목록에서도 핵심적으로 보여야 하나요, 아니면 상세에 들어갔을 때 보여주는 것이 더 맞다고 생각하시나요?

10. 실제 테스트용 Trend 데이터는 어느 수준으로 유지하고 싶으신가요?

Category Master Seed와 Trend 테스트 데이터를 섞어서는 안 됩니다. Category는 기존 Master Data지만 Trend는 계속 바뀌는 동적 데이터이기 때문입니다.

두 가지 방향이 있습니다.

A안 — 테스트 Fixture + 개발용 Demo Seed 분리

```
pytest
→ 각 테스트 Fixture/Factory로 Trend 생성

실제 Swagger / Expo 테스트
→ 별도의 Demo Trend Seed 실행
```

B안 — 공용 Trend Sample Seed

```
개발 DB에
항상 동일한 Trend 샘플 세트를 Seed
```

A는 테스트 독립성이 좋고, B는 팀원이 언제든지 앱을 실행하면 샘플 화면을 볼 수 있다는 장점이 있습니다.

질문: 외부 수집 기능이 생기기 전까지 개발 DB에서도 "샘플 Trend 10~20개가 항상 존재하는 상태"가 편하신가요, 아니면 필요할 때만 별도 Demo Seed를 실행하는 편이 좋으신가요?

4. 이번 First Pass에서 특히 중요한 네 가지

모든 질문이 연결되어 있지만, 우선순위로 보면 ③ 최신의 의미, ② Main과 전체 Trend 화면의 관계, ⑦ Rank 의미, ⑨ AI 요약 노출 위치가 가장 중요합니다.

이 네 가지가 결정되면 나머지는 상당 부분 자연스럽게 따라옵니다. 예를 들어 "최근 관측순 + 무한 스크롤 + Rank 없음"으로 정해지면 `last_collected_at + trend_id` 기반 Cursor Pagination이라는 후보를 구체적으로 검토할 수 있습니다. 반대로 "순위 중심 피드"라면 현재 `trends` 인덱스만 보고 설계해서는 안 되고 `trend_rank_snapshots`의 최신 Snapshot 선정 자체를 먼저 정의해야 합니다.

답변은 1-A, 2-B ...처럼 선택만 하셔도 되고, 애매한 항목은 "모르겠음"이라고 두셔도 좋습니다. 그 다음 단계에서 제가 각 선택의 반례와 트레이드오프를 검토하고, Repository/Service 책임, Response 계약, Frontend Query 구조, Main/TrendCard 책임, Backend/Frontend 테스트 범위까지 하나의 설계로 좁혀가겠습니다.